Analisi Tecnica Completa del Progetto: V2G con LSTM

Autore: Analisi generata da Antigravity AI
Data: Marzo 2026
Progetto: V2G-with-Regression-Learner-and-LSTM-NN

Indice

1. Contesto Applicativo: Vehicle-to-Grid (V2G)
2. Il Problema: Previsione del Consumo Energetico
3. Soluzione: Reti Neurali LSTM
4. Struttura del Dataset
5. Architettura del Codice
6. Pipeline Dati: dal Raw al Modello
7. Architettura della Rete LSTM
8. Ottimizzazione Bayesiana degli Iperparametri
9. Valutazione e Metriche
10. Risultati Sperimentali
11. Analisi Critica e Limitazioni

---

1. Contesto Applicativo: Vehicle-to-Grid (V2G)

Cos'è il V2G?

La tecnologia **Vehicle-to-Grid (V2G)** rappresenta un paradigma emergente nella gestione dell'energia distribuita, in cui i veicoli elettrici (EV) non sono semplici consumatori di energia, ma diventano **partecipanti attivi della rete elettrica**.

In un sistema V2G, la batteria dell'EV può:

- **Caricarsi** dalla rete durante le ore di bassa domanda (notte, festivi) quando l'energia è abbondante e a basso costo.
- **Cedere energia alla rete** durante i picchi di domanda (ore mattutine e serali nei giorni lavorativi), contribuendo alla stabilizzazione della frequenza e della tensione.

Questo crea un sistema bidirezionale che beneficia sia l'utente (riduzione dei costi energetici) sia il gestore della rete (flessibilità e stabilità).

Perché la Previsione del Consumo è Fondamentale in V2G?

Per ottimizzare le strategie di carica/scarica V2G, è indispensabile **conoscere in anticipo** il profilo di consumo energetico delle zone urbane in cui gli EV sono connessi. Questo permette al sistema di:

1. **Pianificare la carica** degli EV nelle fasce orarie a basso carico.
2. **Programmare la restituzione** nelle fasce di picco.
3. **Evitare di sovraccaricare la rete** in momenti già critici.
4. **Massimizzare l'arbitraggio sul prezzo dell'energia** (acquistare a basso costo, vendere ad alto).

Un modello predittivo accurato del consumo energetico è quindi il **nucleo computazionale** di qualsiasi sistema V2G intelligente. L'errore del modello si traduce direttamente in perdita economica o inefficienza gestionale.

Il Contesto Geografico: Roma, 2023

Il progetto lavora su dati reali di consumo energetico di **zone di distribuzione della città di Roma**, anno 2023. Le zone analizzate sono:

ID	Nome	Carattere
8	Zone_1016_Anagnina	Residenziale/Commerciale periferia sud
9	Zone_214_Trieste	Residenziale denso, nord Roma
10	Zone_2004_Della Vittoria/Tor di Quinto	Misto commerciale-residenziale
11	Zone2002_Tor di Quinto	Residenziale

I dati hanno risoluzione **half-hourly** (30 minuti), producendo 48 campioni al giorno per zona.

---

2. Il Problema: Previsione del Consumo Energetico

Tipo di Problema: Regressione su Serie Temporale

Il problema è classificabile come **regressione su serie temporale multivariata** (MTSR - Multivariate Time Series Regression):

- **Input (predittori):** Al tempo *t-1*, la rete riceve un vettore di 9 feature che descrivono le condizioni della mezz'ora appena trascorsa (meteo, ora, giorno, consumo storico).
- **Output (target):** La rete prevede il consumo energetico al tempo *t* corrente — cioè, conoscendo tutto fino a 30 minuti fa, stima il consumo del prossimo step.
- **Modalità:** Seq2Seq (Sequence-to-Sequence) — la rete produce una previsione per ogni step dell'intera sequenza.

Questo è fondamentalmente diverso dal semplice "next-step prediction": nella modalità Seq2Seq, la rete impara a mappare un'intera sequenza di input a un'intera sequenza di output, il che è molto più robusto per previsioni a lungo termine.

Feature Selezionate

Sono stati scelti **9 predittori**, suddivisi in tre categorie:

Variabili meteo (esogene):

- `precipprob` — Probabilità di precipitazione (%)
- `temp` — Temperatura dell'aria (°C)
- `windspeed` — Velocità del vento (km/h)

Variabile calendario:

- `holiday_indicator` — Flag 0/1 per giorni festivi

Variabili cicliche (encoding sinusoidale):

- `hour_sin`, `hour_cos` — Rappresentazione ciclica dell'ora del giorno
- `day_sin`, `day_cos` — Rappresentazione ciclica del giorno della settimana

Variabile di consumo storico:

- `AAC_energy` — Il consumo energetico stesso (kWh), che funge da autoregressor

> ***Nota sul'encoding ciclico:*** *Usare sin/cos invece del numero diretto dell'ora evita l'artefatto per cui l'ora 23 e l'ora 0 sembrano "lontane" numericamente ma sono invece consecutive. Con sin/cos, le 23:30 e le 00:00 sono rappresentate come punti adiacenti su di un cerchio unitario.*

---

3. Soluzione: Reti Neurali LSTM

Perché non un semplice modello di regressione?

Il consumo energetico è una serie temporale con **dipendenze a lungo termine**: il consumo alle 9:00 di lunedì è correlato non solo al consumo delle 8:30 dello stesso giorno, ma anche ai pattern della settimana precedente, ai festivi, alle stagioni. Un modello lineare (regressione, AR, ARMA) cattura solo dipendenze a breve termine e non generalizza bene su pattern complessi non lineari.

Anatomia di una cella LSTM

Le reti **LSTM (Long Short-Term Memory)**, proposte da Hochreiter e Schmidhuber nel 1997, sono un tipo di **rete ricorrente** (RNN) progettate per ricordare informazioni su sequenze molto lunghe. La chiave del meccanismo è la **cella di memoria** regolata da tre gate:

1. **Forget Gate** (`f_t = σ(W_f · [h_{t-1}, x_t] + b_f)`):
   Decide quanta informazione del passato (`h_{t-1}`) va dimenticata. Un'uscita vicina a 0 significa "dimentica tutto", vicina a 1 "ricorda tutto".

2. **Input Gate** (`i_t = σ(W_i · [h_{t-1}, x_t] + b_i)`):
   Decide quante delle nuove informazioni dall'input corrente vanno memorizzate nella cella.

3. **Output Gate** (`o_t = σ(W_o · [h_{t-1}, x_t] + b_o)`):
   Controlla quale parte dello stato della cella viene esposta come output `h_t` .

La **cella di stato** `C_t` si aggiorna come:

```
C_t = f_t ⊙ C_{t-1} + i_t ⊙ tanh(W_C · [h_{t-1}, x_t] + b_C)
```

Questo meccanismo "a cancello" risolve il problema del **vanishing gradient** che affligge le RNN classiche, permettendo alla rete di mantenere informazioni per centinaia di step temporali.

Modalità Seq2Seq (Sequence-to-Sequence)

Nel progetto viene usata la modalità **Seq2Seq** invece della più semplice "one-to-one":

- **One-to-one:** Dato X[1..T], prevedi solo Y[T+1]. Produce una sola previsione per sequenza.
- **Seq2Seq:** Dato X[1..T], prevedi Y[2..T+1] — un output per ogni step dell'input. La rete interpreta **ogni timestep** come una previsione del timestep successivo.

Il vantaggio del Seq2Seq è che **ogni sequenza di training produce T coppie di (input, label)** invece di una sola, rendendo l'apprendimento molto più ricco con pochi dati disponibili.

In MATLAB, questo si ottiene impostando `OutputMode = "sequence"` nel `lstmLayer` , che fa sì che il layer restituisca l'output `h_t` per **ogni** timestep `t` , non solo l'ultimo.

---

4. Struttura del Dataset

Raw Data Sources

Il dataset è composto da tre sorgenti distinte:

1. Dati di consumo per zona (`/Dati Estratti (from Condivisione)/`)

- `Zone1016_new.mat` → variabili: `AAC_energy` , `time_vector`
- `Zone214_new.mat`
- `Zone2004_new.mat`
- `Zone2002_new.mat`

Ogni file contiene un vettore temporale (`datetime`) e un vettore dei consumi energetici in kWh, a risoluzione di 30 minuti per un anno intero.

2. Dati meteorologici (`/Gabriele Datas/metero_year_hh.mat`)

- Struttura tabellare (`meteo_year_hh`) con colonne: `temp` , `precipprob` , `windspeed` , e altre variabili meteo.
- Frequenza: 30 minuti, sincronizzata con i dati di consumo.

3. Calendario Festività (`/Gabriele Datas/holidays.mat`)

- Vettore booleano `vacanze` (0 = giorno ordinario, 1 = festivo).
- Include domeniche, festività nazionali italiane.

Struttura Temporale Reale

I dati **non coprono l'intero anno 2023** in modo continuo. Sono presenti 4 "cluster" temporali (finestre di dati disponibili), con gap di settimane tra uno e l'altro:

```
Cluster 1: [15 Feb 2023] → [28 Feb 2023]   (≈14 giorni)
Cluster 2: [31 Mag 2023] → [15 Giu 2023]   (≈15 giorni)
Cluster 3: [12 Lug 2023] → [25 Lug 2023]   (≈14 giorni)
Cluster 4: [27 Set 2023] → [10 Ott 2023]   (≈14 giorni)
```

Questo è un **vincolo fondamentale** dell'intero progetto: disponiamo di circa 56 giorni di dati totali, frammentati in 4 finestre stagionali (inverno, primavera/estate, estate piena, autunno). Questo copre stagioni diverse, ma rende impossibile l'apprendimento di pattern annuali o mensili.

---

5. Architettura del Codice

Il progetto è organizzato in due script principali e 14 funzioni di supporto modulari.

```
V2G-with-Regression-Learner-and-LSTM-NN/
├── LSTM.m                    ← Entry point: training e valutazione
├── LSTM_Predictions.m        ← Previsione cross-zona con modello salvato
├── scripts/
│   ├── loadZoneData.m        ← Caricamento e fusione dati
│   ├── selectRepresentativeDays.m  ← Selezione automatica giorni val/test
│   ├── splitTrainValTest.m   ← Split del dataset in train/val/test
│   ├── normalizeZScore.m     ← Normalizzazione z-score anti-leakage
│   ├── normalizeZScoreFull.m ← Normalizzazione per dataset completo
│   ├── createLstmSequences.m ← Costruzione sequenze Seq2Seq
│   ├── getBestIndicators.m   ← Estrazione metriche da `trainnet` info
│   ├── computeMetrics.m      ← Calcolo RMSE, R², MAE, MAPE
│   ├── saveModels.m          ← Salvataggio persistente del modello
```

```
│   ├── plotResults.m         ← Visualizzazione real vs predicted
│   ├── splitTrainTest.m      ← Split semplificato train/test
│   ├── createRegressionLags.m  ← (per Regression Learner, non LSTM)
│   └── plotRegressionResults.m ← (per Regression Learner, non LSTM)
└── docs/
    └── analisi_progetto.md   ← Questo documento
```

---

6. Pipeline Dati: dal Raw al Modello

La pipeline di preparazione dei dati è la parte più critica del progetto. Segue esattamente questi passi in `LSTM.m`:

Passo 1 — Caricamento (`loadZoneData`)

```
datas = loadZoneData(root, zoneId);
```

La funzione:

1. Seleziona il file `.mat` corretto per la `zoneId` richiesta.
2. Carica i dati meteo dal file condiviso `metero_year_hh.mat`.
3. Carica il vettore ferie da `holidays.mat`.
4. **Fonde tutto in un'unica tabella** MATLAB (`table`) con colonne: `time_vector`, `temp`, `precipprob`, `windspeed`, `holiday_indicator`, `AAC_energy`.

Passo 2 — Selezione Giorni Val/Test (`selectRepresentativeDays`)

```
[valDays, testDays] = selectRepresentativeDays(datas);
```

Questo è il componente più sofisticato del preprocessing. I motivi per cui è necessario:

Problema: Con soli ~56 giorni, scegliere giorni di test "a caso" potrebbe portare a una valutazione distorta (es. tutti i giorni di test in estate, mentre il training è in inverno).

Algoritmo:

1. Individua i **4 cluster** di dati (gap > 7 giorni = nuovo cluster).
2. Per ciascun cluster, estrae:
   - Il **giorno normale più tipico** (non-domenica, più vicino alla mediana energetica del cluster).
   - La **domenica più tipica** (domenica con profilo meteo e consumo più prossimo alla mediana domenicale del cluster).
3. Per scegliere il "giorno più tipico", usa un **filtro a due stadi**:
   - Primary: seleziona i 5 giorni con consumo totale più vicino alla mediana.
   - Secondary (tie-breaker): tra questi 5, sceglie quello con profilo meteo (temp, precipitazione, vento) più normale.
   - **Vincolo:** Il giorno selezionato deve avere anche il giorno precedente (t-1) disponibile nel dataset (necessario per il regressor).
4. Assegna le coppie alternandole tra validation e test.

Risultato: **2 giorni per validation** (1 normale + 1 domenica) e **2 giorni per test** (1 domenica + 1 normale), ognuno proveniente da cluster stagionali diversi.

Passo 3 — Split Train/Val/Test (`splitTrainValTest`)

```
[trainData, valData, testData] = splitTrainValTest(datas, valDays, testDays);
```

I giorni selezionati come val/test vengono **rimossi** dal dataset principale. Tutto il resto diventa training set. In questo modo:

- `trainData` : ~52 giorni × 48 campioni = ~2496 righe
- `valData` : 2 giorni × 48 campioni = ~384 righe (+ giorno regressor per ciascuno)
- `testData` : 2 giorni × 48 campioni = ~384 righe (+ giorno regressor per ciascuno)

Passo 4 — Feature Engineering (Encoding Ciclico)

Dopo lo split, `LSTM.m` aggiunge manualmente le feature cicliche:

```
hour_rad = hour(time_vector) * (2*pi/24) + minute(time_vector) * (2*pi/(24*60));
trainData.hour_sin = sin(hour_rad);
trainData.hour_cos = cos(hour_rad);
```

Stesso procedimento per `day_sin` / `day_cos` (su 7 giorni). Questo trasforma l'ora e il giorno della settimana da valori discreti circolari in due coordinate sul piano (come latitudine e longitudine su un cerchio).

Passo 5 — Normalizzazione Z-Score (`normalizeZScore`)

```
[trainNorm, testNorm, normParams] = normalizeZScore(trainData, testData, columnsToNorm);
```

Principio anti-data-leakage: La media (μ) e la deviazione standard (σ) vengono calcolate **esclusivamente sul training set**. Questi stessi parametri vengono poi applicati ai set di validazione e test. Questo è fondamentale perché:

Se normalizzassimo usando la statistica sull'intero dataset, il modello potrebbe "vedere" informazioni future durante l'addestramento (tipico errore di data leakage che gonfia artificialmente le metriche).

La funzione gestisce anche il caso σ = 0 (feature costante) sostituendo con σ = 1, evitando divisioni per zero.

Il risultato è che ogni feature ha media 0 e deviazione standard 1 rispetto alle statistiche di training. I `normParams` vengono salvati nel modello per la fase di denormalizzazione.

Passo 6 — Costruzione Sequenze Seq2Seq (`createLstmSequences`)

```
[xTrain, yTrain, timeVectorTrain] = createLstmSequences(trainNorm, predictors, target);
```

Questa funzione implementa la logica Seq2Seq:

1. Trova i **break point** nel training set (dove i dati non sono più contigui, gap > 35 minuti).
2. Per ogni blocco contiguo, crea una coppia (X, Y) dove:
   - `X = data[1:end-1, :]` → Input per ogni step (tutte le 9 feature)
   - `Y = data[2:end, :]` → Target per ogni step (solo `AAC_energy` , il consumo al passo successivo)
3. Ogni coppia diventa un elemento del cell array `xTrain` / `yTrain` .

Risultato diagnostico tipico:

```
xTrain:       7 sequences | shape: [143×9]
xValidation:  2 sequences | shape: [95×9]
xTest:        2 sequences | shape: [95×9]
```

Le 7 sequenze di training derivano dai ~52 giorni divisi in blocchi contigui (i 4 cluster spezzati dalla rimozione dei giorni val/test creano discontinuità aggiuntive).

---

7. Architettura della Rete LSTM

Struttura dei Layer

Seguendo le linee guida MathWorks per il Time Series Forecasting:

```
sequenceInputLayer(9)             % Input: 9 feature per timestep
    ↓
lstmLayer(119, OutputMode="sequence")  % 119 hidden units, output su tutta la seq.
    ↓
fullyConnectedLayer(100)          % Bottleneck: compressione rappresentazione
    ↓
reluLayer()                       % Non-linearità: attivazione ReLU
    ↓
dropoutLayer(0.33)                % Regularizzazione: dropout 33%
    ↓
fullyConnectedLayer(1)            % Output: valore scalare (consumo previsto)
```

Perché questo layout specifico:

- **`sequenceInputLayer(9)`:** Riceve 9 feature per ogni timestep. `Normalization="none"` perché normalizziamo già manualmente.
- **`lstmLayer(119)`:** 119 unità nascoste, trovate ottimali dalla Bayesian Optimization. `OutputMode="sequence"` è essenziale per il Seq2Seq: permette alla rete di emettere `h_t` ad ogni `t`, non solo alla fine.
- **`fullyConnectedLayer(100)`:** Layer "bottleneck" aggiunto secondo le specifiche MathWorks. Combina le 119 rappresentazioni LSTM in 100 feature dense prima dell'output.
- **`reluLayer()`:** Introduce non-linearità. Il ReLU (Rectified Linear Unit) è economico da calcolare e non soffre di vanishing gradient.
- **`dropoutLayer(0.33)`:** Durante il training, il 33% dei neuroni viene disattivato a caso ad ogni iterazione. Forza la rete a non dipendere da un singolo neurone, riducendo l'overfitting. Non attivo durante l'inferenza.
- **`fullyConnectedLayer(1)`:** Output finale: un singolo valore normalizzato che rappresenta il consumo energetico previsto al timestep successivo.

Opzioni di Training

```
options = trainingOptions("adam",
    MaxEpochs           = 200,
    MiniBatchSize       = 30,
    InitialLearnRate    = 0.01,
    LearnRateSchedule   = "piecewise",
    LearnRateDropFactor = 0.2,
```

```
    LearnRateDropPeriod = 50,
    GradientThreshold   = 1,
    SequencePaddingDirection = "left",
    Shuffle             = "every-epoch",
    ValidationPatience  = 20
)
```

Punti chiave:

- **Adam Optimizer:** Algoritmo adattivo che combina momentum e RMSProp. Converge significativamente più velocemente di SGD per serie temporali.
- **Learning Rate = 0.01:** Valore alto per convergere in fretta (200 epoche sono poche). Il scheduler lo riduce del 80% ogni 50 epoche (× 0.2), garantendo una fine-tuning precisa.
- **GradientThreshold = 1:** Clipping del gradiente. Se il gradiente supera la norma 1, viene ridimensionato. Previene esplosioni del gradiente nelle LSTM profonde.
- **SequencePaddingDirection = "left":** Le sequenze di lunghezza diversa vengono allineate a destra (padding a sinistra con zeri). Questo è corretto per serie temporali: lo zero-padding iniziale ha poca influenza sull'output finale.
- **ValidationPatience = 20:** Early stopping: se la validation loss non migliora per 20 epoche consecutive il training si ferma automaticamente.

---

8. Ottimizzazione Bayesiana degli Iperparametri

Motivazione

Con 5 iperparametri liberi e un training time di ~5-10 secondi per configurazione, una gridsearch esaustiva su griglie di 10 valori richiederebbe `10^5 = 100.000` training. La **Bayesian Optimization** risolve questo problema in modo elegante.

Funzionamento

L'ottimizzazione Bayesiana costruisce un **modello probabilistico** (surrogate model, tipicamente un Gaussian Process) dello spazio degli iperparametri. Ad ogni trial:

1. Usa il surrogate model per identificare dove è **più probabile** trovare un valore ottimale.
2. Esegue il training con quella configurazione.
3. Aggiorna il surrogate model con il nuovo punto osservato.

La strategia di acquisizione `expected-improvement-plus` bilancia **esplorazione** (provare zone sconosciute dello spazio) con **sfruttamento** (concentrarsi attorno ai minima già trovati).

Spazio di Ricerca Configurato

Parametro	Range	Tipo
`numHiddenUnits`	[50, 300]	Intero
`numLayers`	[1, 3]	Intero
`initialLearnRate`	[1e-4, 1e-2]	Reale (log-scale)
`dropoutRate`	[0.0, 0.5]	Reale
`miniBatchSize`	[16, 128]	Intero

La funzione obiettivo è la **RMSE di validazione** calcolata su 50 epoche (versione compressa del training per velocità), e viene minimizzata dall'algoritmo.

Risultati: Convergenza dopo 100 Trial

Dalla seconda esecuzione a 100 trial, i risultati mostrano:

Rank	Hidden Units	Layers	LR	Dropout	Batch	RMSE Val
1	**119**	**1**	**0.01000**	**0.333**	**30**	**0.3019**
2	138	1	0.00993	0.184	72	0.3072
3	90	1	0.00981	0.087	33	0.3101

Pattern convergenti chiari:

- `numLayers = 1` vince sistematicamente. Con 7 sole sequenze di training, più layer aumentano solo l'overfitting. La semplicità è ottimale.
- `initialLearnRate ≈ 0.01` — Il valore massimo del range. Con 200 epoche, un learning rate alto è necessario per convergere.
- `numHiddenUnits ≈ 100-140` — Zona di ottimo stabile. Sotto i 70 il modello underfitta, sopra i 200 overfitting.
- `dropoutRate` : meno determinante (0.09 e 0.49 danno risultati simili). Il dropout non è il parametro critico.
- `miniBatchSize ≈ 17-35` — Batch piccoli permettono aggiornamenti frequenti e miglior generalizzazione con poche sequenze.

---

9. Valutazione e Metriche

Metriche di Qualità del Modello

R^2 (Coefficiente di Determinazione)

```
R² = 1 - SS_res / SS_tot
```

Varia da 0 a 1 (idealmente). Indica la percentuale di varianza del target spiegata dal modello. Un $R^2 = 0.90$ significa che il modello spiega il 90% della variabilità del consumo.

RMSE (Root Mean Square Error)
Errore quadratico medio nella stessa unità del target (kWh). Penalizza fortemente gli errori grandi (outlik). Utile per comparare modelli.

MAE (Mean Absolute Error)
Errore medio assoluto in kWh. Più robusto agli outlier rispetto a RMSE.

MAPE (Mean Absolute Percentage Error)
Errore percentuale medio. **Fortemente sconsigliato** quando il target può essere zero o quasi zero: `MAPE = |err| / |reale|` tende a infinito per reali vicini a zero. Nel dataset, ci sono istanti con consumi quasi nulli (domeniche, notte), rendendo il MAPE intrinsecamente instabile.

Baseline di Persistenza

La vera domanda non è "il LSTM funziona?", ma **"il LSTM è meglio dell'alternativa più semplice?"**

Il **modello di persistenza** è la baseline: `y_hat(t) = y(t-1)` — prevedi che al prossimo step il consumo sia uguale a quello corrente. È incredibilmente semplice ma spesso sorprendentemente accurato su serie stazionarie a breve termine.

Se l'LSTM non batte la persistenza, il modello non apporta valore.

Analisi degli Errori Implementate in `LSTM.m`

Oltre alle metriche aggregate, `LSTM.m` include quattro analisi diagnostiche:

1. **Analisi Residui nel Tempo** — scatter plot degli errori `(y_real - y_pred)` nel tempo. Pattern sistematici indicano feature mancanti.
2. **Distribuzione dei Residui** — istogramma. Idealmente campana centrata sullo zero (errori gaussiani).
3. **Autocorrelazione dei Residui (ACF)** — Se i residui sono autocorrelati, la rete non ha catturato tutto il pattern temporale.
4. **Q-Q Plot** — Verifica della normalità dei residui. Utile per giustificare assunzioni parametriche.

E due analisi aggiuntive: 5. **Breakdown errore per ora** — identifica le fasce orarie critiche. 6. **Top 10 Outliers** — mostra i 10 campioni con errore più alto, con metadati climatici e temporali per diagnosi manuale.

---

10. Risultati Sperimentali

Modello Allenato su Zona 8 (Anagnina)

Addestrato con: Bayesian Optimization (100 trial), iperparametri ottimali: HU=119, L=1, LR=0.01, D=0.33, BS=30.

Set	R^2	RMSE (kWh)
Training (norm.)	0.9020	—
Validation (norm.)	0.9106	—
Test (denorm.)	0.7929	56.74 kWh
Persistenza (test)	0.7998	55.90 kWh

Osservazioni: Il modello sul test set è marginalmente sotto la persistenza per la Zona 8. Tuttavia, il cross-testing rivela che **il modello generalizza eccellentemente** ad altre zone:

Generalizzazione Cross-Zona (modello Zona 8):

Zona	R^2 LSTM	RMSE LSTM	R^2 Persistenza	Superiore?
Zona 8	0.9103	69.90 kWh	0.8455	✅ +6.5%
Zona 9	0.7902	55.80 kWh	0.7409	✅ +4.9%
Zona 10	0.8160	58.96 kWh	0.7758	✅ +4.0%
Zona 11	0.8623	92.53 kWh	0.8361	✅ +2.6%

Il modello **batte sempre la persistenza** su tutte le zone, anche su quelle mai viste in training.

Modello Allenato su Zona 9 (Trieste)

Addestrato con: Iperparametri di default (post-Bayesian): HU=119, L=1, LR=0.01, D=0.33, BS=30.

Set	R^2	RMSE (kWh)
Training (norm.)	0.8241	—
Validation (norm.)	0.8026	—
Test (denorm.)	0.6742	60.87 kWh
Persistenza (test)	0.7273	55.70 kWh

Osservazioni: La Zona 9 mostra un R^2 inferiore sul test set e non batte la persistenza in quel contesto ristretto. Tuttavia, nel cross-testing sull'intero dataset di 58 giorni, il modello raggiunge R^2=0.79 superando la persistenza.

Generalizzazione Cross-Zona (modello Zona 9):

Zona	R^2 LSTM	RMSE LSTM	R^2 Persistenza	Superiore?
Zona 8	0.8530	89.48 kWh	0.8455	✅ +0.75%
Zona 9	0.7898	55.85 kWh	0.7409	✅ +4.9%
Zona 10	0.7849	63.76 kWh	0.7758	✅ +0.9%
Zona 11	0.8434	98.68 kWh	0.8361	✅ +0.7%

Pattern degli Errori per Ora

In entrambi i modelli emerge un pattern sistematico:

- **Errore massimo:** Fascia 8-17h (ore lavorative, picco mattutino e pomeridiano).
- **Errore minimo:** Fascia 3-4h (notte, consumo quasi costante, facile da prevedere).

Questo è tipico delle zone urbane commerciali-residenziali: il brusco cambio tra consumo notturno piatto e picco del mattino è intrinsecamente difficile da prevedere con dati meteo e calendario, perché è influenzato dal comportamento umano (orari di lavoro, abitudini).

---

11. Analisi Critica e Limitazioni

Punti di Forza del Progetto

1. **Pipeline dati robusta:** La normalizzazione anti-leakage, la selezione deterministica dei giorni rappresentativi e la modalità Seq2Seq sono tutti implementati correttamente.
2. **Ottimizzazione Bayesiana efficace:** 100 trial in ~18 minuti per trovare iperparametri ottimali in uno spazio 5-dimensionale.
3. **Generalizzazione cross-zona:** Il modello supera la persistenza su **tutte e 4 le zone**, incluse zone mai viste durante training. Ciò indica che la rete ha imparato pattern energetici urbani generali, non specifici di una singola zona.
4. **Architettura MathWorks standard:** Il design segue le best practice ufficiali nei tutorial Time Series Forecasting.

Limitazioni Identificate

1. **Dataset limitato (56 giorni):** Con solo 7 sequenze di training, la rete ha poche informazioni per generalizzare. Un dataset annuale completo darebbe risultati nettamente superiori.
2. **MAPE instabile:** La metrica MAPE non è affidabile su questo dataset per la presenza di consumi quasi nulli (domeniche, notti festive). È preferibile concentrarsi su RMSE e MAE denormalizzate.
3. **Picco mattutino sistematico:** L'errore nelle fasce 8-12h è costantemente il doppio dell'errore medio. Possibili mitigazioni: feature aggiuntive (tipologia di giorno lavorativo, festivi locali), o modelli separati per fascia oraria.
4. **Test set ristretto (2 giorni):** Valutare un modello su soli 2 giorni introduce alta varianza statistica. Il cross-testing su 58 giorni è molto più rappresentativo.
5. **Inferenza aperta:** Il LSTM è addestrato in modalità Seq2Seq ma viene valutato in "open loop" (ogni previsione usa le feature reali, non le previsioni precedenti). Questo è appropriato per una previsione "what-if" ma non per una previsione autonoma multi-step.

Possibili Estensioni Future

- **Più dati:** Raccolta di dati sull'intero anno per almeno 2-3 anni. Obiettivo: 10.000+ sequenze di training.
- **Attention Mechanism:** Aggiungere un layer di Self-Attention sopra l'LSTM (architettura Transformer-LSTM ibrida) per catturare meglio dipendenze a lungo termine.
- **Previsione multi-step:** Passare da previsioni al passo successivo (t+1) a previsioni su orizzonti più lunghi (es. 4 ore = 8 step) per una pianificazione V2G più realistica.
- **Integrazione prezzi energia:** Aggiungere il prezzo dell'energia come feature esogena per ottimizzare le strategie di carica/scarica V2G.

Fine del documento. Ultima revisione: Marzo 2026.